

A decorative geometric pattern on the left side of the slide. It features a dark blue background with various shapes and patterns: a white circle in the top left, a grey semi-circle, a pink and blue striped triangle, a pink square with a white grid pattern, a grey triangle, and a pink triangle. A white diagonal line runs from the top left to the bottom right.

DAG SCHEDULER, STAGES & TASKS (WITH PYSPARK)

WHAT IS A DAG (DIRECTED ACYCLIC GRAPH)?

DAG is a logical execution plan of Spark jobs .Spark builds the DAG from RDD/DataFrame lineage, DAG defines how data flows from input to result

📌 Note: DAG is built before execution, ensuring lazy evaluation

What is a DAG?

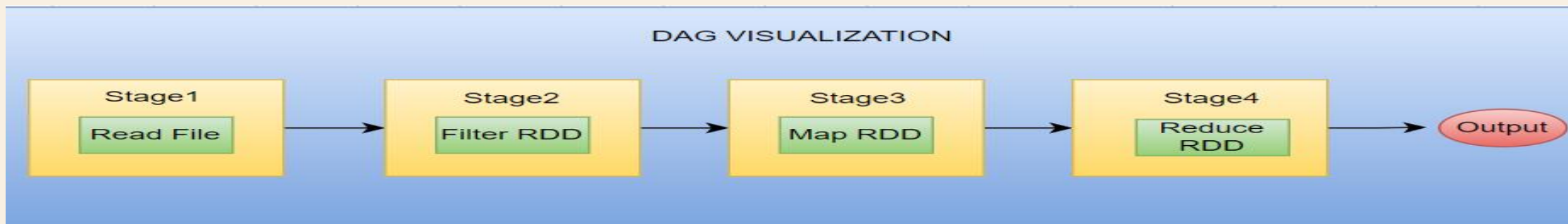
- Directed: Operations are executed in a specific order.
- Acyclic: There are no loops or cycles in the execution plan.

Role of DAG in Spark

- Logical Execution Plan: When you submit a Spark application, Spark translates the high-level operations (like transformations and actions) into a DAG of stages and tasks.
- Stages and Tasks: Each stage depends on the completion of the previous stage, and each task within a stage can run independently of the others.

How DAG Works

1. Transformations: These are lazy operations (e.g., `filter`, `map`) that build up the DAG. They do not trigger execution immediately.
2. Actions: These operations (e.g., `collect`, `show`) trigger the execution of the DAG.



Role of DAG

The DAG Scheduler is responsible for:

- 1.Creating Stages:** It converts the logical execution plan (lineage) of transformations into a DAG of stages.
- 2.Pipelining Transformations:** Groups transformations that can be executed together into a single stage.
- 3.Handling Failures:** Ensures fault tolerance by recomputing lost partitions.
- 4.Optimizing Execution:** Attempts to minimize shuffles and optimize execution.

How the DAG Scheduler Works

- 1. Logical Plan:** When you define transformations on a DataFrame or RDD, Spark creates a logical plan that outlines the sequence of transformations.
- 2. DAG Creation:** The DAG Scheduler converts this logical plan into a physical execution plan, breaking it into stages.
 - 1. Stages:** Each stage contains a set of transformations that can be executed together without requiring a shuffle.
 - The DAG breaks the job down into a sequence of stages, where each stage represents a group of tasks that can be executed independently of each other. The tasks within each stage can be executed in parallel across the machines.
 - 3. Improve Efficiency:** The DAG allows Spark to perform various optimizations, such as pipelining, task reordering, and pruning unnecessary operations, to improve the efficiency of the job execution.
- 3. Task Creation:** Within each stage, the DAG Scheduler creates tasks, which are the smallest units of work to be executed by the executors.
- 4. Task Scheduling:** The Task Scheduler then assigns these tasks to executors based on data locality and resource availability.
- 5. Execution:** Executors run the tasks, process the data, and store the results.
- 6. Actions and Triggers:** An action triggers the execution of the DAG. For example, calling `collect()`, `save()`, or `count()` on a DataFrame or RDD.

Components of DAG

1. Stages: A stage represents a set of tasks that can be executed in parallel. There are two types of stages in Spark: shuffle stages and non-shuffle stages. Shuffle stages involve the exchange of data between nodes, while non-shuffle stages do not.

2. Tasks: A task represents a single unit of work that can be executed on a single partition of an RDD. Tasks are the smallest units of parallelism in Spark.

3. Dependencies: The dependencies between RDDs determine the order in which tasks are executed. There are two types of dependencies in Spark: narrow dependencies and wide dependencies. Narrow dependencies indicate that each partition of the parent RDD is used by at most one partition of the child RDD, while wide dependencies indicate that each partition of the parent RDD can be used by multiple partitions of the child RDD.

4. Job

- **Definition:** A job is triggered by an action (e.g., `collect`, `show`).
- **Execution:** A job is divided into stages by the DAG Scheduler.

5. DAG Scheduler

- **Role:** Divides a job into stages and tasks, and schedules them for execution.
- **Function:** Manages task execution, retries, and fault tolerance.

EXAMPLE

```
from pyspark.sql import SparkSession
```

Initialize Spark session

```
spark = SparkSession.builder.appName("DAG Example").getOrCreate()
```

Load data from multiple sources

```
df1 = spark.read.csv("hdfs://path/to/file1.csv", header=True, inferSchema=True)
```

```
df2 = spark.read.csv("hdfs://path/to/file2.csv", header=True, inferSchema=True)
```

```
df3 = spark.read.jdbc(url="jdbc:oracle:thin:@//host:port/service", table="table_name", properties={"user": "username",  
"password": "password"})
```

Perform complex transformations

```
df1_filtered = df1.filter(df1["age"] > 30)
```

```
df2_filtered = df2.filter(df2["salary"] > 50000)
```

```
joined_df = df1_filtered.join(df2_filtered, df1_filtered["id"] == df2_filtered["id"]).drop(df2_filtered["id"])
```

```
aggregated_df = joined_df.groupBy("department").avg("salary")
```

Further transformation with a third dataset

```
final_df = aggregated_df.join(df3, aggregated_df["department"] == df3["department"]).select(aggregated_df["*"],  
df3["extra_info"])
```

Trigger action

```
result = final_df.collect()
```

Stop Spark session

```
spark.stop()
```

DAG Scheduler Breakdown for Above Example

1.Logical Plan Creation:

- Loading `df1`, `df2`, and `df3` creates initial logical plans for each dataset.
- `df1_filtered`, `df2_filtered`, `joined_df`, `aggregated_df`, and `final_df` define a series of transformations, forming a complex logical plan.

2.DAG Construction:

- Stage 1:** Load `df1` and filter it (`df1_filtered`).
- Stage 2:** Load `df2` and filter it (`df2_filtered`).
- Stage 3:** Join `df1_filtered` and `df2_filtered`, then group by department and calculate the average salary (`aggregated_df`).
- Stage 4:** Load `df3` and join it with `aggregated_df`, selecting the final columns (`final_df`).

3.Task Creation:

- Each stage is divided into tasks based on the partitions of the data.For example, if `df1` is partitioned into 4 parts, Stage 1 will have 4 tasks.

4.Task Scheduling:

- Tasks are scheduled to run on executors, considering data locality to reduce data shuffling.
- Executors run the tasks for each stage.

5.Execution:

- Stage 1:** `df1` is loaded and filtered. The results are stored in memory or disk.
- Stage 2:** `df2` is loaded and filtered. The results are stored.
- Stage 3:** The join operation requires shuffling data based on the join key, creating a shuffle boundary. The grouped and aggregated results are stored.
- Stage 4:** `df3` is loaded, joined with `aggregated_df`, and the final result is computed.

6.Action Trigger:

- The `collect()` action triggers the execution of the entire DAG.
- The results are collected back to the driver.

Logical Plan:

df1 ➡ filter ➡ df1_filtered

df2 ➡ filter ➡ df2_filtered

df1_filtered + df2_filtered ➡ join ➡ joined_df

joined_df ➡ groupBy + avg ➡ aggregated_df

aggregated_df + df3 ➡ join ➡ final_df

DAG of Stages:

Stage 1:

[Load df1, filter]

Stage 2:

[Load df2, filter]

Stage 3:

[Join df1_filtered and df2_filtered, groupBy, avg]

Stage 4:

[Load df3, join with aggregated_df, select final columns]

Tasks:

- Stage 1: [Task 1, Task 2, Task 3, Task 4] (based on partitions of df1)
- Stage 2: [Task 1, Task 2, Task 3, Task 4] (based on partitions of df2)
- Stage 3: [Task 1, Task 2, Task 3, Task 4] (shuffle, based on join key)
- Stage 4: [Task 1, Task 2, Task 3, Task 4] (based on partitions of df3)

Execution:

- Stage 1 tasks -> Stage 2 tasks -> Shuffle -> Stage 3 tasks -> Stage 4 tasks

